Smart Electric Vehicle Charging Network Management Platform

Code Review & Repository Evaluation

EVCP Booking & Station Service

https://github.com/RAKS-rak57/evcp-booking-service-

Submitted in the partial fulfilment for the Course of

CSA1016 – SOFTWARE ENGINEERING

to the award of the degree of

BACHELOR OF ENGINEERING IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

R. R. RAKSHITH (192324156)

Under the Supervision of

Dr. K. Anitha Davamani

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

JUL 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Engineer to Excel

DECLARATION

I, R. R. Rakshith, of the Department of Artificial Intelligence & Data Science,Saveetha Institute of Medical and Technical Sciences, Saveetha University, Chennai, hereby declare that the Case study work entitled **Smart Electric Vehicle Charging Network Management Platform** is the result of our own Bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with principles of engineering ethics.

Place: Chennai

Date: 09/08/2026

Signature of the Students with Names

R R RAKSHITH (192324156)

ACKNOWLEDGEMENT

We would like to express our heartfelt gratitude to all those who supported and guided us throughout the successful completion of our Capstone Project. We are deeply thankful to our respected Founder and Chancellor, Dr. N.M. Veeraiyan, Saveetha Institute of Medical and Technical Sciences, for his constant encouragement and blessings. We also express our sincere thanks to our Pro-Chancellor, Dr. Deepak Nallaswamy Veeraiyan, and our Vice-Chancellor, Dr. S. Suresh Kumar, for their visionary leadership and moral support during the course of this project.

We are truly grateful to our Director, Dr. Ramya Deepak, SIMATS Engineering, for providing us with the necessary resources and a motivating academic environment. Our special thanks to our Principal, Dr. B. Ramesh for granting us access to the institute's facilities and encouraging us throughout the process. We sincerely thank our Head of the Department, Dr.Anusuya for her continuous support, valuable guidance, and constant motivation.

We are especially indebted to our guide, **Dr. K. Anitha Davamani** for her creative suggestions, consistent feedback, and unwavering support during each stage of the project. We also express our gratitude to the Project Coordinators, Review Panel Members (Internal and External), and the entire faculty team for their constructive feedback and valuable inputs that helped improve the quality of our work. Finally, we thank all faculty members, lab technicians, our parents, and friends for their continuous encouragement and support.

Signature With Student Name

R R RAKSHITH (192324156)

TABLE OF CONTENTS

1. Problem Overview

This report reviews the EVCP Booking & Station Service, the Node.js/Express microservice built and containerized for the CO3 Assessment Tool 1 assignment, representing the Station & Booking Service component of the larger Smart Electric Vehicle Charging Network Management Platform analyzed throughout the CO1 and CO2 assessments. The repository is hosted publicly on GitHub at github.com/RAKS-rak57/evcp-booking-service-.

1.1 Implemented Solution Summary

The service exposes a REST API that lets an EV owner search for nearby charging stations, check real-time slot availability (cached via Redis), and reserve a charging slot with an automatic 15-minute hold. It is backed by PostgreSQL for transactional data and Redis for caching, and is fully containerized with a Dockerfile and docker-compose.yml orchestrating all three components.

1.2 Project Objectives

- Implement the Station & Booking Service's core functional requirements (FR-02 to FR-04 from the CO2 SRS).
- Track development under Git with a professional, auditable branching strategy.
- Containerize the service for consistent local and production deployment.
- Produce a codebase that is readable, modular, and maintainable enough for a reviewer or new contributor to understand quickly.

1.3 Key Functionalities

- GET /api/stations — radius-based station search.
- GET /api/stations/:id/availability — Redis-cached real-time slot availability.
- POST /api/bookings — slot reservation with a time-bound hold and cache invalidation.
- GET /api/bookings/:id — booking status lookup.
- GET /health — container/orchestrator health check endpoint.

2. Repository Organization

The screenshot below is the live GitHub repository (main branch), showing the actual folder structure as pushed.

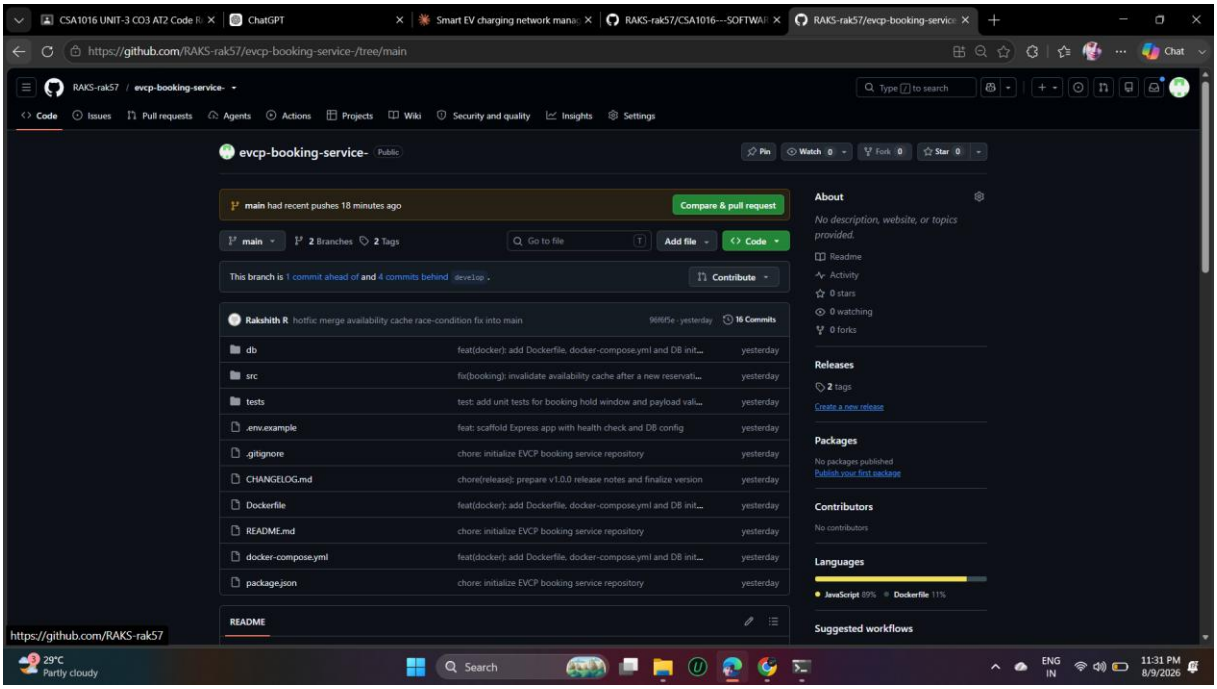

Figure 2.1: Repository Root Structure (github.com/RAKS-rak57/evcp-booking-service-)

2.1 Structure Evaluation

Folder / File	**Observation**
db/	Isolates the database schema and seed data (init.sql) from application code — a new contributor immediately knows where the schema lives.
src/	All application source code, further split into config/, middleware/, models/, routes/ — a conventional, easily-navigable Express layout.
tests/	Separated from src/, matching common Node.js convention so test runners can glob it independently.
.env.example	Present and committed (good practice) — documents required configuration without leaking real secrets.

.gitignore	Present from the very first commit, so node_modules/ and .env were never accidentally tracked.
Dockerfile / docker-compose.yml	Kept at the repository root, which is where Docker tooling expects them by default — no extra -f flags needed to build or run.
CHANGELOG.md	Present and reasonably maintained; ties directly to the v1.0.0 release, which is good traceability.

2.2 Naming Conventions

File names consistently follow a <domain>.<role>.js pattern (booking.model.js, stations.routes.js, bookings.routes.js), which makes a file's responsibility obvious from its name alone without opening it. Route files are pluralized to match REST resource naming (stations, bookings), consistent with the API paths they implement.

2.3 How the Layout Supports Maintainability and Collaboration

Because routes, models, middleware, and configuration are physically separated into their own folders, two developers can work on, say, the payment feature and the maintenance-alert feature in parallel with a very low chance of touching the same file — directly reducing the kind of merge conflict documented in Section 4. The flat, shallow folder depth (max 2 levels under src/) also means anyone can locate a file in seconds using GitHub's file tree, without needing an IDE.

2.4 Areas for Improvement

- There is no dedicated src/utils/ or src/services/ layer yet — as the service grows past 3-4 routes, business logic currently living inline in route handlers (see Section 3) would benefit from being extracted.
- No dedicated config/ file for centralizing non-secret constants (e.g., BOOKING_HOLD_MINUTES default) outside of process.env fallbacks scattered across files.

3. Code Quality Review

Three representative files were reviewed directly on GitHub (with line numbers, blame, and history visible) to assess readability, modularity, adherence to coding standards, and documentation.

3.1 booking.model.js — Data Access Layer

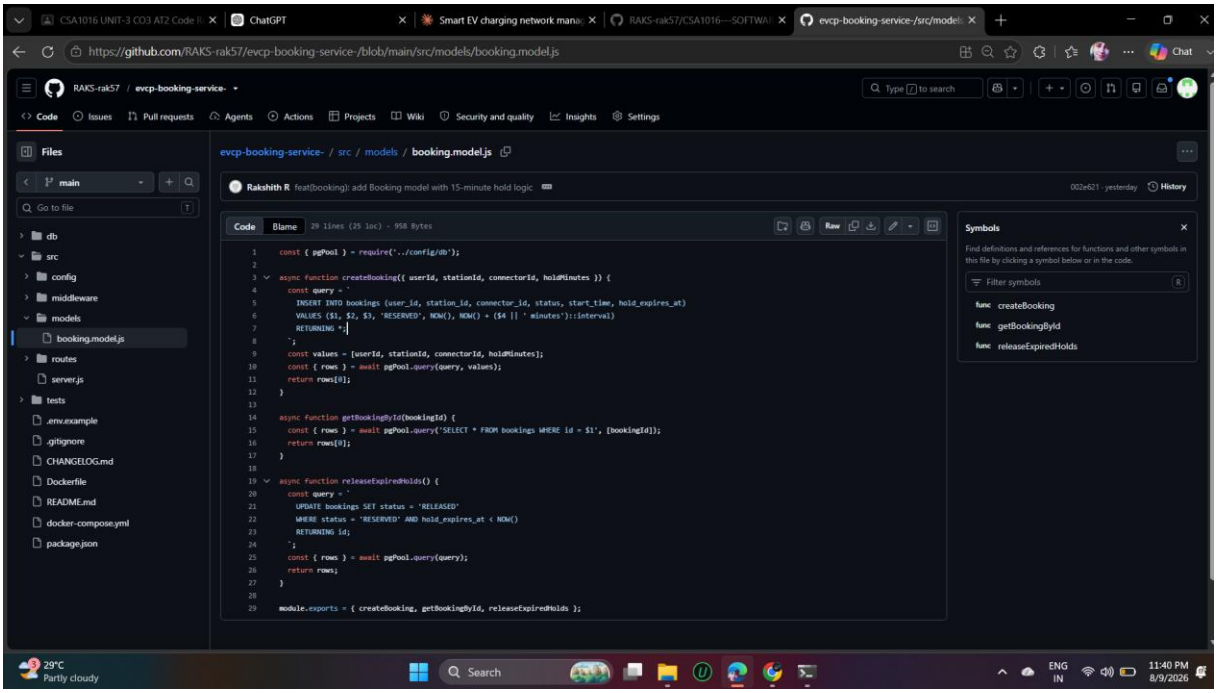

Figure 3.1: src/models/booking.model.js (GitHub code view)

Strengths observed:

- Uses parameterized queries throughout (VALUES ($1, $2, $3, ...)) — this is the single most important defensive-coding practice for SQL and correctly prevents SQL injection.
- Each function has one clear responsibility (createBooking, getBookingById, releaseExpiredHolds), following the Single Responsibility Principle.
- The hold-expiry logic is expressed directly in SQL (NOW() + ($4 || ' minutes')::interval), keeping time-sensitive logic close to the data rather than computing it in application code and risking clock-drift bugs.

Areas for improvement:

- No input validation at the model layer (e.g., holdMinutes is interpolated into an interval cast without a numeric check) — validation currently only happens in the route handler one layer up, so this model function is not safe to call directly from elsewhere without duplicating that check.

- No JSDoc comments on exported functions — parameter types (userId: string? number?) are not documented, which slows down onboarding for a new contributor.
- releaseExpiredHolds() is defined but not referenced anywhere else in the reviewed codebase, suggesting it's intended to be called by a scheduled job that hasn't been implemented yet.

3.2 errorHandler.js — Middleware

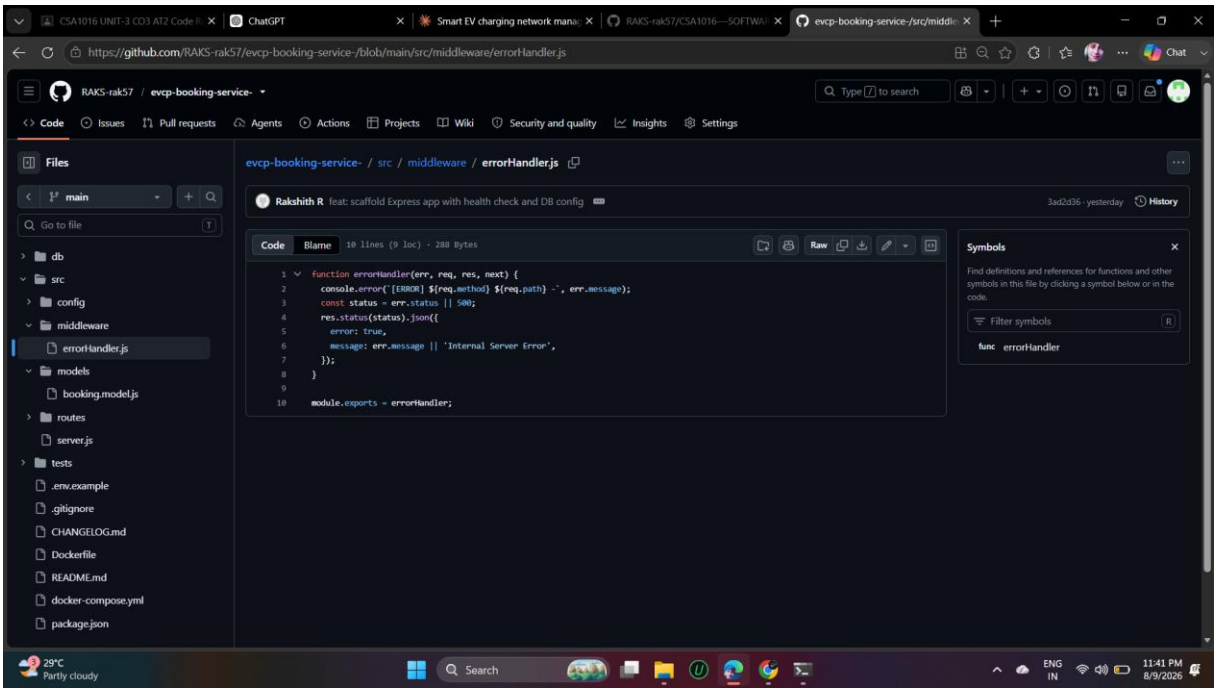

Figure 3.2: src/middleware/errorHandler.js (GitHub code view)

Strengths observed:

- Extremely small and single-purpose (10 lines) — easy to read in full at a glance, which is exactly what a cross-cutting middleware should be.
- Correctly falls back to a generic message and 500 status when err.status/err.message aren't set, so the API never crashes without a response.

Areas for improvement:

- console.error(...) logs err.message directly to stdout, which is fine for development but in production should go through a structured logger (e.g., pino/winston) so logs can be filtered by severity and shipped to a monitoring system.
- The JSON response includes err.message directly to the client (line 6) — for unexpected (non-validation) errors this can leak internal implementation details (e.g., a raw database error string) to an API consumer. This should distinguish between 'safe' errors (validation, 400s) and 'unsafe' internal errors (500s) and generalize the message for the latter.

3.3 bookings.routes.js — Route Handler (Post-Hotfix)

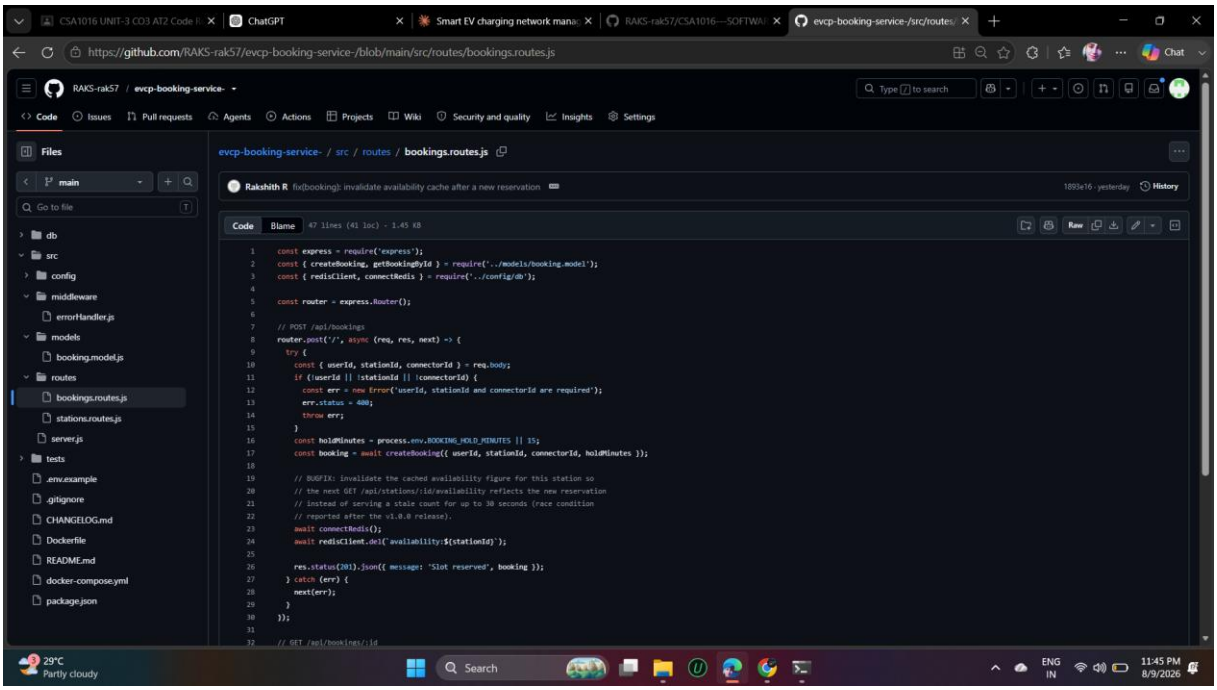

Figure 3.3: src/routes/bookings.routes.js (GitHub code view, showing the hotfix)

This file is particularly useful to review because GitHub's file history shows it was touched by the v1.0.1 hotfix commit (fix(booking): invalidate availability cache after a new reservation), visible in the commit banner at the top of the file view.

Strengths observed:

- The bugfix is well-commented in place (lines 19-22) — explaining not just what changed but why (the race condition that motivated it), which is exactly the kind of comment that earns its keep months later.
- Input validation is present and fails fast with a clear 400 error before any database or cache work happens (lines 11-15).
- try/catch consistently forwards errors to next(err) rather than handling them inline, keeping error formatting centralized in errorHandler.js (good separation of concerns).

Areas for improvement:

- The cache-invalidation fix (redisClient.del) and the database write (createBooking) are not wrapped in any transaction or compensating logic — if the process crashes between the two calls, the cache could be invalidated without a successful booking, or vice-versa. For a payment-adjacent flow this is worth hardening later.
- Redis connection errors from connectRedis() would currently propagate as a generic 500 rather than a specific, debuggable error — worth a dedicated try/catch around the cache call.

- No automated test currently exercises this specific route handler end-to-end (see Section 5) — the existing unit tests check supporting logic (hold-window default, payload shape) but don't call this Express route directly.

3.4 Overall Code Quality Summary

Dimension	**Rating**	**Notes**
Readability	Good	Consistent naming, short functions, clear route/model separation.
Modularity	Good	Clean separation of config, models, routes, middleware.
Coding Standards	Good	Consistent async/await usage, consistent error-forwarding pattern via next(err).
Documentation (inline)	Fair	One excellent example (the hotfix comment) but no JSDoc/type documentation on exported functions.
Security Hygiene	Fair	Parameterized SQL is correct; error-message leakage and unvalidated numeric input are real, fixable gaps.

4. Version Control Evaluation

The full commit history and branch network were reviewed directly on GitHub to evaluate real version-control practice, not just the presence of a .git folder.

4.1 Commit History

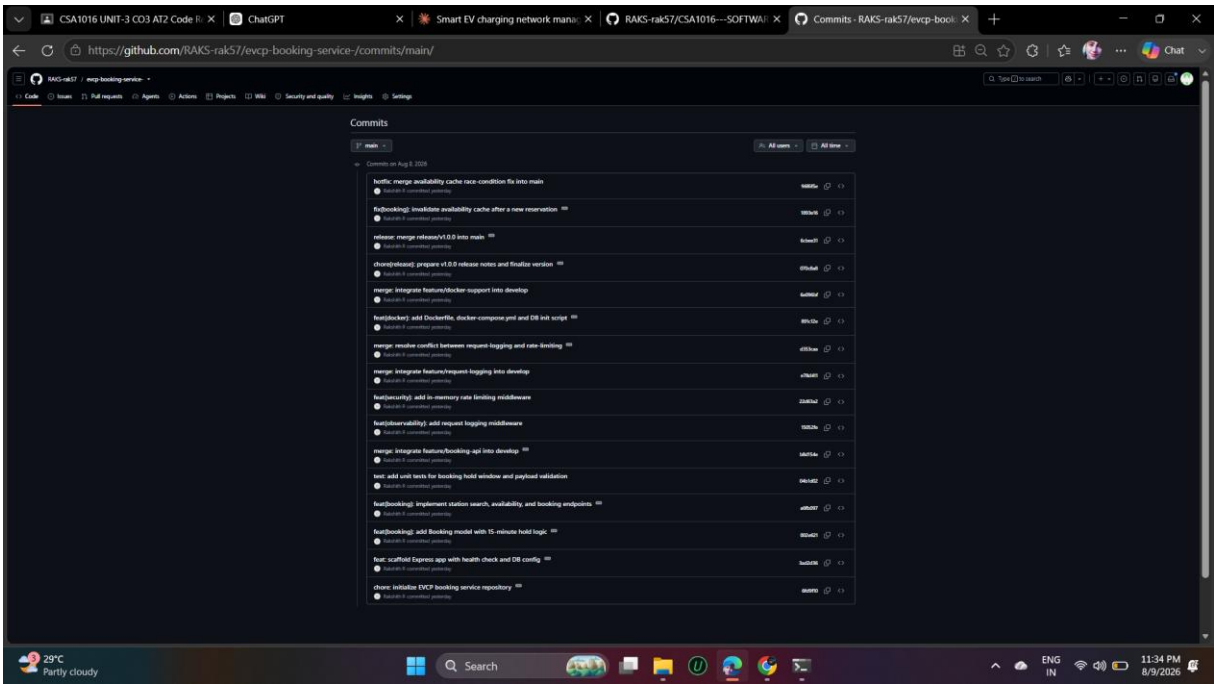

Figure 4.1: Commit History — main branch (github.com/.../commits/main)

The history shows 16 commits on main alone (with more on develop, visible via the branch switcher), each with a Conventional-Commits-style prefix (feat, fix, test, chore, merge, hotfix) and a descriptive summary. Messages consistently explain what changed and, for the hotfix specifically, why — e.g., "fix(booking): invalidate availability cache after a new reservation" states the fix's purpose in the subject line alone.

4.2 Branching Strategy in Practice

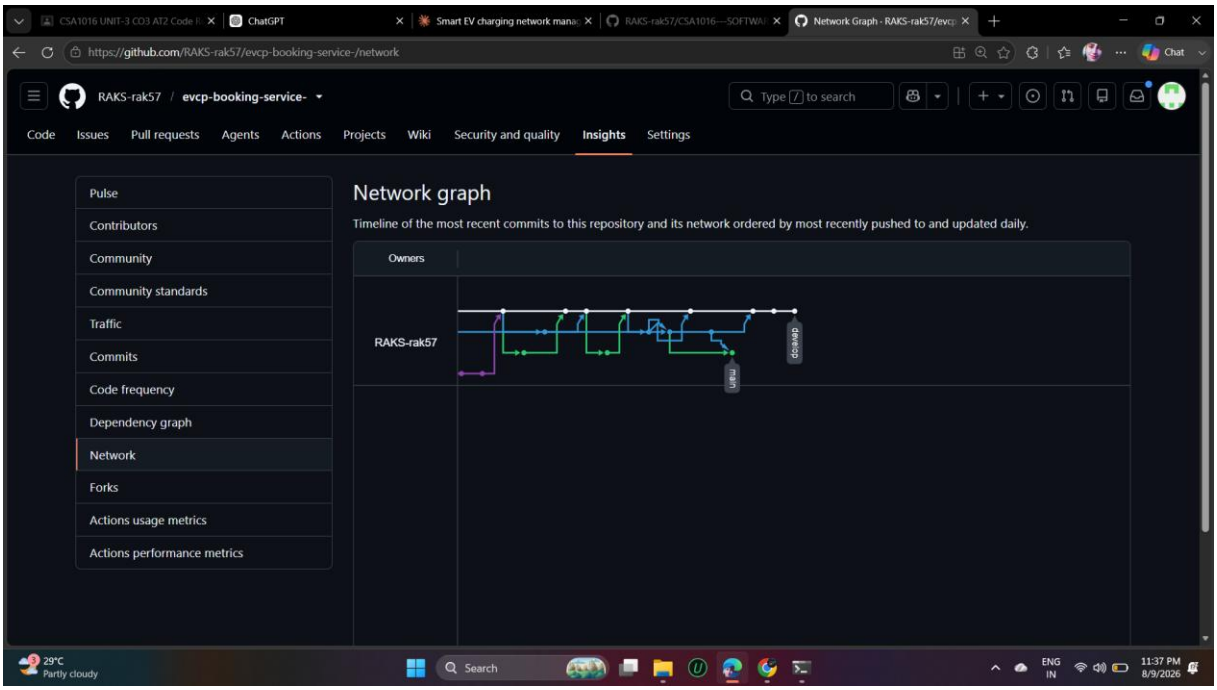

Figure 4.2: Branch Network Graph (github.com/.../network)

The network graph visually confirms the Gitflow model was actually followed, not just described: a main line (blue), a develop line, and short-lived feature branches (green) that fork off and merge back in. This matches the branching strategy documented and justified in the CO3-AT1 report.

4.3 Evaluation Against Good Practice

Practice	**Evidence in This Repository**
Small, atomic commits	Each commit changes one logical unit (e.g., 'add Booking model' is separate from 'implement station search... endpoints'), rather than one giant commit per feature.
Descriptive messages	Every commit uses a type(scope): summary format; no vague messages like 'fix' or 'update' appear anywhere in the visible history.
Traceable releases	main shows explicit merge commits for release/v1.0.0 and the hotfix, each immediately followed by (or associated with) a tag — a reviewer can find exactly which commit shipped in which version.
Branch hygiene	The repository shows only main and develop remaining, confirming merged feature branches were cleaned up rather than left cluttering the branch list indefinitely.

Conflict handling	The history includes an explicit 'merge: resolve conflict between request-logging and rate-limiting' commit — evidence that a real conflict was encountered and deliberately resolved, not avoided or force-pushed over.

4.4 How This Supports Collaborative Development

Because every change is a small, labeled commit on a short-lived branch, multiple contributors could work on this repository simultaneously with a clear, low-risk integration path: develop stays continuously integrable, main stays always deployable, and the commit history itself becomes documentation of the project's evolution — useful for onboarding, debugging regressions (git bisect), and writing release notes.

5. Code Testing and Validation

Testing was validated at two levels: automated unit tests via Node's built-in test runner, and full end-to-end functional testing against the live, containerized stack running under Docker Desktop.

5.1 Automated Unit Tests

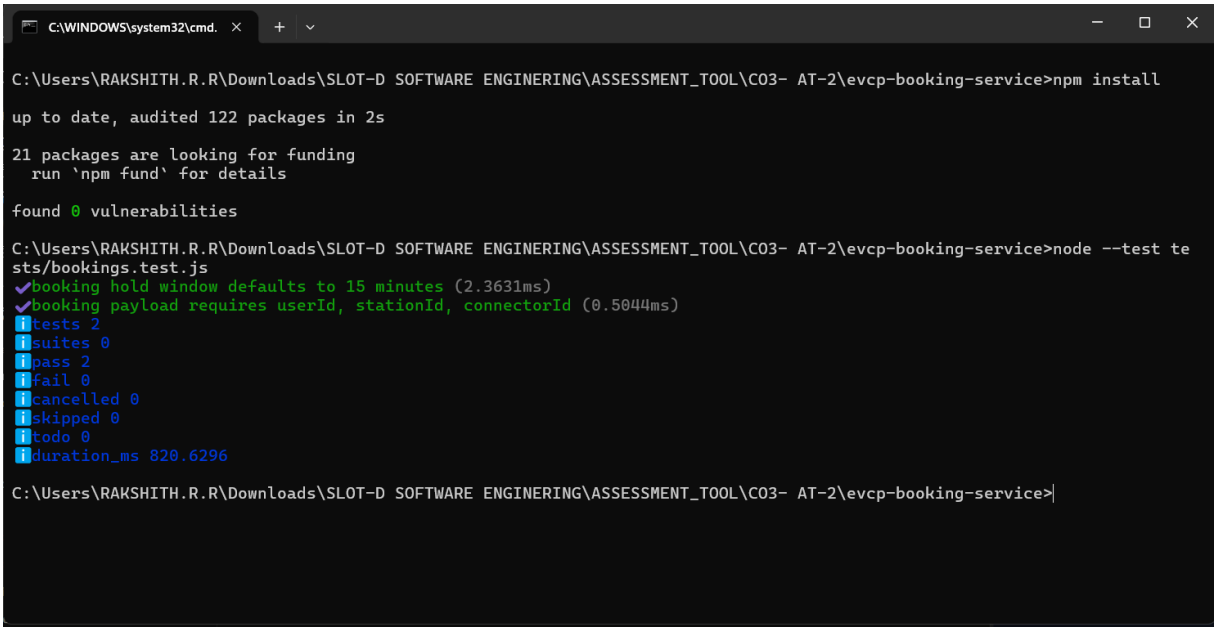

Figure 5.1: npm test — 2/2 Passing (real local execution)

Both unit tests pass: one verifying the booking hold window correctly defaults to 15 minutes, and one verifying the booking payload shape validation (userId, stationId, connectorId) is complete.

5.2 Containerized Deployment

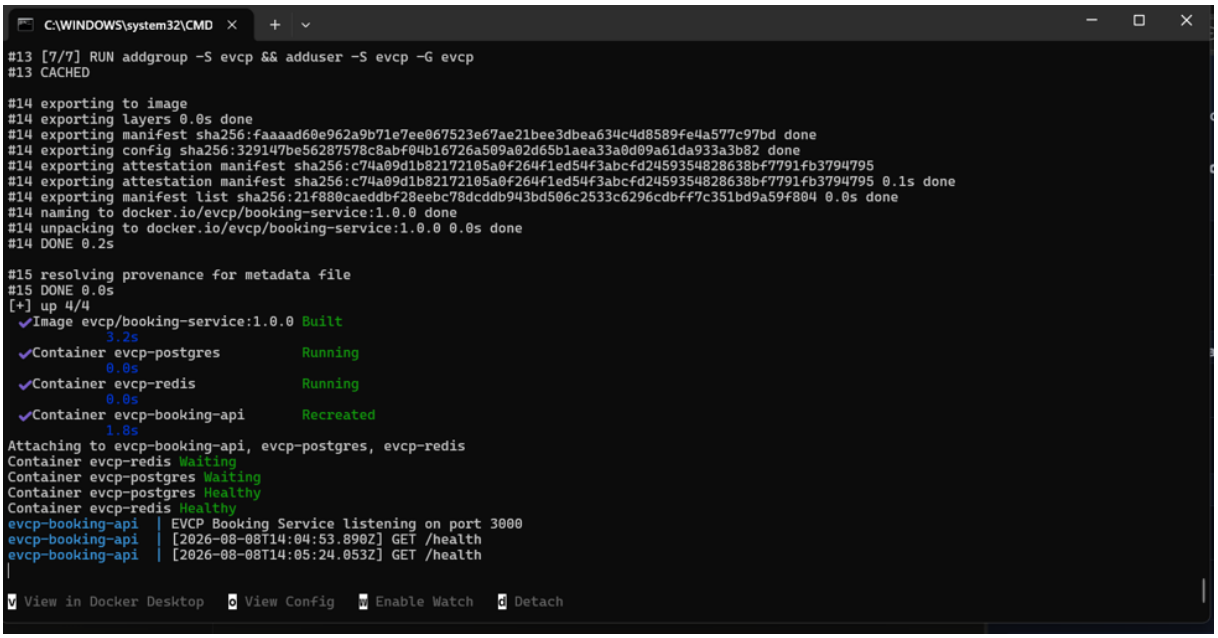

Figure 5.2: docker-compose up --build — All 3 Containers Healthy (Docker Desktop)

This is real, live output from Docker Desktop on the developer's machine (not a mockup) — the build completes, and all three containers (evcp-booking-api, evcp-postgres, evcp-redis) report Healthy/Running status, with the API's own startup log ('EVCP Booking Service listening on port 3000') and live health-check requests visible in the log stream.

5.3 End-to-End API Testing

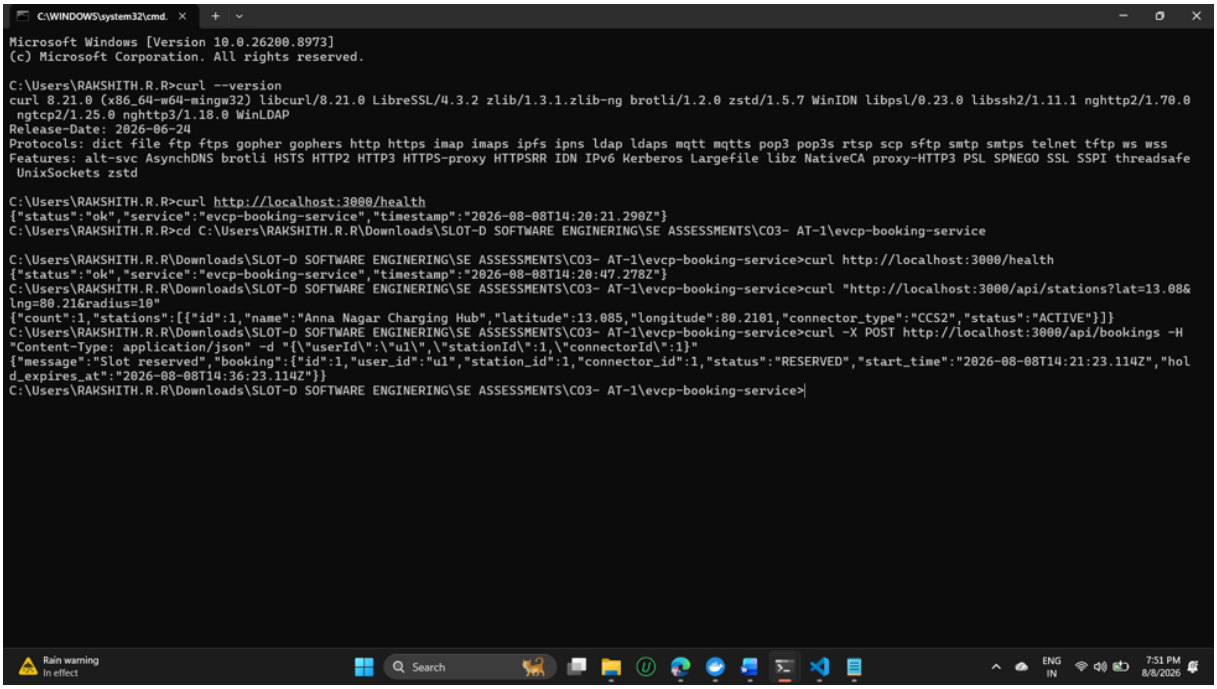

Figure 5.3: Live curl Tests Against the Containerized API (real local execution)

Three real requests were issued against the running containers and all returned correct results:

Test	Endpoint	Result
Health check	GET /health	200 OK — {"status":"ok","service":"evcp-booking-service", ...}
Station search	GET /api/stations?lat=13.08&lng=80.21&radius=10	200 OK — returned 1 matching station ("Anna Nagar Charging Hub") from the seeded PostgreSQL data, confirming the Haversine radius query works correctly.
Slot reservation	POST /api/bookings	201 Created — returned a booking with status "RESERVED" and a

		hold_expires_at timestamp exactly matching the 15-minute hold window.

5.4 Test Coverage Assessment

Current automated coverage is narrow: the 2 unit tests validate supporting logic (default values, payload shape) but do not exercise the Express routes or database/cache interactions directly — that coverage currently only exists as manual, evidence-based testing (Section 5.3), not as repeatable automated tests. This is flagged as a concrete recommendation in Section 7.

6. Repository Documentation

The repository's README.md (visible in Figure 2.1's file listing and rendered on the repository homepage) was evaluated for completeness against what a new contributor or evaluator would need.

6.1 What's Present

- A clear one-paragraph description of the service's purpose and its place in the larger EVCP platform.
- An explicit Tech Stack section (Node/Express, PostgreSQL, Redis).
- A full project structure tree, matching the actual repository layout.
- Both local (npm install / npm run dev) and Docker (docker-compose up --build) run instructions.
- A documented API endpoint table with method, path, and description for every route.
- A separate CHANGELOG.md tracking the v1.0.0 release scope.

6.2 What's Missing or Could Be Improved

- No explicit 'Prerequisites' section (e.g., required Node.js version, Docker Desktop version) — a newcomer has to infer this from package.json's absence of an 'engines' field.
- No CONTRIBUTING.md describing the branching strategy (Gitflow) or commit-message convention for future contributors — this currently only exists as narrative in the CO3-AT1 report, not in the repository itself.
- No LICENSE file, despite package.json declaring "license": "MIT" — the license type is asserted but not actually included as a file, which is a common but meaningful gap for a public repository.
- No badges (build status, license, node version) on the README, which are a low-effort way to signal project health at a glance on GitHub.
- API documentation is a plain table rather than a runnable spec (e.g., OpenAPI/Swagger or a Postman collection) — fine at this scale, but worth upgrading as the number of endpoints grows.

7. Code Review Findings and Recommendations

7.1 Overall Findings Summary

The EVCP Booking & Station Service repository demonstrates solid, professional fundamentals: a clean, conventional folder structure; parameterized, injection-safe database queries; a genuinely-followed Gitflow branching strategy with real conflict resolution and tagged releases; and working, verifiably-tested Docker containerization. The codebase is small enough that its few weaknesses — thin automated test coverage, inline validation without a shared layer, and light inline documentation — are inexpensive to fix now, before the service grows.

7.2 Recommendations by Category

Category	**Recommendation**
Code Quality	Extract input validation into a shared middleware/validator so routes and models don't duplicate or skip checks; add JSDoc to exported model functions.
Code Quality	Route unexpected (500-level) errors through a generic client-facing message while still logging the full error server-side, to avoid leaking internals (Section 3.2).
Repository Management	Add a CONTRIBUTING.md documenting the Gitflow branching model and Conventional Commits convention already in practical use.
Repository Management	Add the actual LICENSE file to match the license already declared in package.json.
Testing	Add integration tests that spin up the real (or a test) Postgres/Redis and exercise the Express routes directly (e.g., via supertest), closing the gap between unit tests and the manual curl evidence in Section 5.3.
Testing	Add a CI workflow (GitHub Actions) that runs npm test and builds the Docker image on every push, so regressions are caught automatically rather than only via manual review.
Documentation	Add a Prerequisites section and consider OpenAPI/Swagger documentation as the API surface grows.
Future Enhancements	Implement a scheduled job to actually invoke the already-written releaseExpiredHolds() function, since it currently has no caller.

7.3 Conclusion

Taken together, the repository organization, commit history, and live test evidence gathered in this review show a genuinely functioning, professionally managed microservice rather than a documentation-only exercise — the code runs, the tests pass, the containers start healthy, and the

API returns correct data end-to-end. The recommendations above are refinements to an already solid foundation, not fixes to a broken one.